ECE 385
Spring 2025

Experiment #3

A logic Processor
Lab-3

Cristina Natalia Lumban Tobing 3220300398
Artha Amanda Sidauruk 3220300399
TA: Qiyang Wu

Table of Contents

Introduction 3

High-level Function, operations, and bits 3

Pre-lab Answers 3

Pre-lab Question A: Describe the simplest (two-input one-output) circuit that can optionally invert a signal (i.e., one input determines if the output is equal to the other input or equal to the other input inverted). Sketch your circuit. 3

Pre-lab Question B: Explain how a modular design such as that presented above improves testability and cuts down development time. 3

Operation of Logic Processor 4

The sequence of switches the user must flip to load data into the A and B registers. 4

The sequence of switches the user must flip to initiate a computation and routing operation. 4

Written description, block diagram, and state machine diagram of the logic processor Written description 5

Block Diagram (High-level Block Diagram) 7

State Machine Diagram 7

Design steps taken and detailed circuit schematic diagram. 8

Written procedure of the design steps taken 8

Circuit Schematic 11

Description of all bugs encountered, and corrective measures taken 13

Conclusion 14

Lab summarizes 14

Post-lab answer 14

Difficulties you had in debugging the circuit. How does the modular approach help you isolate design and wiring faults? 14

What are the tradeoffs of a Mealy machine vs. a Moore machine? 14

Introduction

High-level Function, operations, and bits

In this lab, we are learning to design and build our own bit-serial logic operation processor. We built a 4-bit logical processor that is able to carry out bitwise operations on 2 four bit string operands. These processors are able to AND, NAND, OR, NOR, XNOR, XOR, etc. This system is a digital circuit designed to take in two 4-bit strings, perform a logical operation on them (the type of which depends on user input), and store the result in a user-chosen register. It uses a 4-bit serial shifting technique to handle the binary data, performs one operation at a time, and waits for further commands to continue.

Pre-lab Answers

Pre-lab Question A: Describe the simplest (two-input one-output) circuit that can optionally invert a signal (i.e., one input determines if the output is equal to the other input or equal to the other input inverted). Sketch your circuit.

The simplest (two-input one-output) circuit that can optionally invert a signal (i.e., one input determines if the output is equal to the other input or equal to the other input inverted) is by using a XNOR or a XOR gate. Since (for XOR) if the select input it 0, then the other value would be passed through, and when the select input is 1, then the other input pin would be inverted at the output.

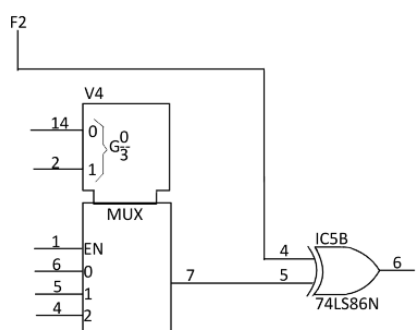

Figure 1: Sample Circuit

Pre-lab Question B: Explain how a modular design such as that presented above improves testability and cuts down development time.

It enables the developer to design and test each individual component or unit separately. This approach leads to faster debugging, as issues can be tackled at a more manageable level. Debugging a small circuit with a single function is much simpler than trying to troubleshoot a large, complex circuit responsible for numerous tasks. Additionally, this method enhances testability by allowing the developer to verify the output of each unit individually, ensuring that each part performs correctly. This is much more efficient than testing the entire circuit at once and then backtracking to identify and resolve any errors.

Operation of Logic Processor

The sequence of switches the user must flip to load data into the A and B registers.

To load data into registers A and B, start by ensuring all switches are turned off and the shift signal in the Mealy machine is set to its lowest level. There are four switches corresponding to the data bits D3-D0 for each register—one set for register A and another for register B. Once the user selects the data, it must be manually entered by setting the load switch for the respective register to high. The data bits D3-D0 are arranged in little-endian order, where D3 is the most significant bit and D0 is the least significant bit.

For example, to load the D3-D0 data into Register A, set the load signal for A to high while keeping E low. This will load the data D3-D0 into Register A. The process for loading data into Register B is identical. Additionally, both load switches for A and B can be set to high if you want to load the data into both registers simultaneously during the same clock cycle.

The sequence of switches the user must flip to initiate a computation and routing operation.

The logic processor can perform 8 different processes, and one of these processes can be selected by setting the bits F2, F1, and F0. To initiate the computation, the Execute switch must be flipped to signal that the processor should start executing the chosen process. Once activated, the computation and register shifting operations are managed by the Finite State Machine (FSM) of the Control Unit, as illustrated in the figure.

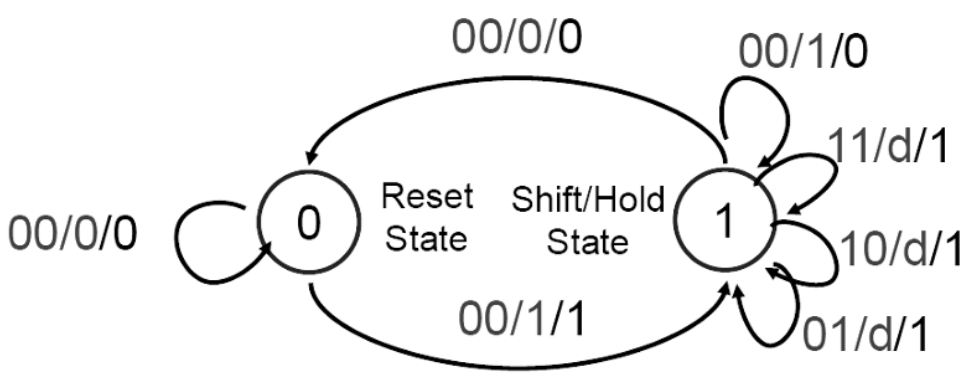

Figure 2: FSM from lab 3 manual

Once the user has selected the process to compute, they manually set the bits F2, F1, and F0 using switches to correspond to the desired process. After these bits are configured, the switches

for F2, F1, and F0 remain in the "on" position. Next, the user must determine which register will hold the result of the computation. This is done by selecting the bits R1 and R0 in the routing unit. The routing unit will then determine whether each register will retain its initial value, store the computed value, or hold the value from the other register. The status of these decisions will be indicated by the LEDs connected to the system. Once the user has made their choice, they manually adjust the switches for R1 and R0 to set the routing configuration.

Function Selection Inputs			Computation Unit Output
F2	F1	F0	f(A, B)
0	0	0	A AND B
0	0	1	A OR B
0	1	0	A XOR B
0	1	1	1111
1	0	0	A NAND B
1	0	1	A NOR B
1	1	0	A XNOR B
1	1	1	0000

Table1: Computational Unit

Routing Selection		Router Output	
R1	R0	A*	B*
0	0	A	B
0	1	A	F
1	0	F	B
1	1	B	A

Table 2: Routing Unit

Written description, block diagram, and state machine diagram of the logic processor

Written description

a) Register Unit
The register unit consists of 2 shift registers, each receiving a shift signal from the control unit, which determines whether the data should be shifted. One shift register holds the value of register A, while the other holds the value of register B. Custom values can be loaded separately into each of these shift registers. When the Execute signal is active and the shift signal is enabled, the shift registers will shift their data out serially to the computation unit.

b) Computation Unit

The computation unit is made up of various logic gates to perform the different bitwise operations required by the user. It receives two values from the A and B registers, and based on the user's selection, it determines which operation to execute. The unit includes a 4:1 multiplexer (MUX) and an XNOR gate, which together enable the implementation of the 8 different operation choices available to the user. These components allow for flexibility in performing a wide range of logical operations on the input data.

c) Routing Unit

The routing unit receives three different values: A, B, and F (the output from the computation unit). It is also connected to the shift registers, enabling the shifting of values into them based on the user's choice. The unit includes a 4:1 multiplexer (MUX) with two user inputs, allowing the user to decide which value to store in the shift registers A and B. The user can choose to store either A, B, or F into the registers, with the available options being AB, AF, FB, or BA. This flexibility allows the user to control how the data is routed into the shift registers.

d) Control Unit

The control unit functions as a finite state machine that manages whether the shift registers should continue shifting data. It takes in three inputs: load A, load B, and execute. When either load A or load B is set to 1, the corresponding shift register (A or B) is loaded with the value from the DIN switches. Once the execute signal is set to 1, the control unit allows one computation to occur and ensures that the process stops after 4 bits have been shifted in and out of the shift register. The control unit determines when to stop shifting.

To simplify the design and reduce debugging time, we chose to use a 741S169 shift register, which helps manage the state transitions and timing for the shifting operations.

In summary, the system operates with the Control Unit managing the overall timing and flow, controlling when to execute calculations and whether to shift or load data into the shift registers. The Computation Unit is governed by a simple operation code, F2-F0, which specifies the type of calculation to be performed. Similarly, the Routing Unit uses the operation code R1-R0 to determine the different routing methods for data. This structured approach allows the user to efficiently control the operations and data flow throughout the system.

Block Diagram (High-level Block Diagram)

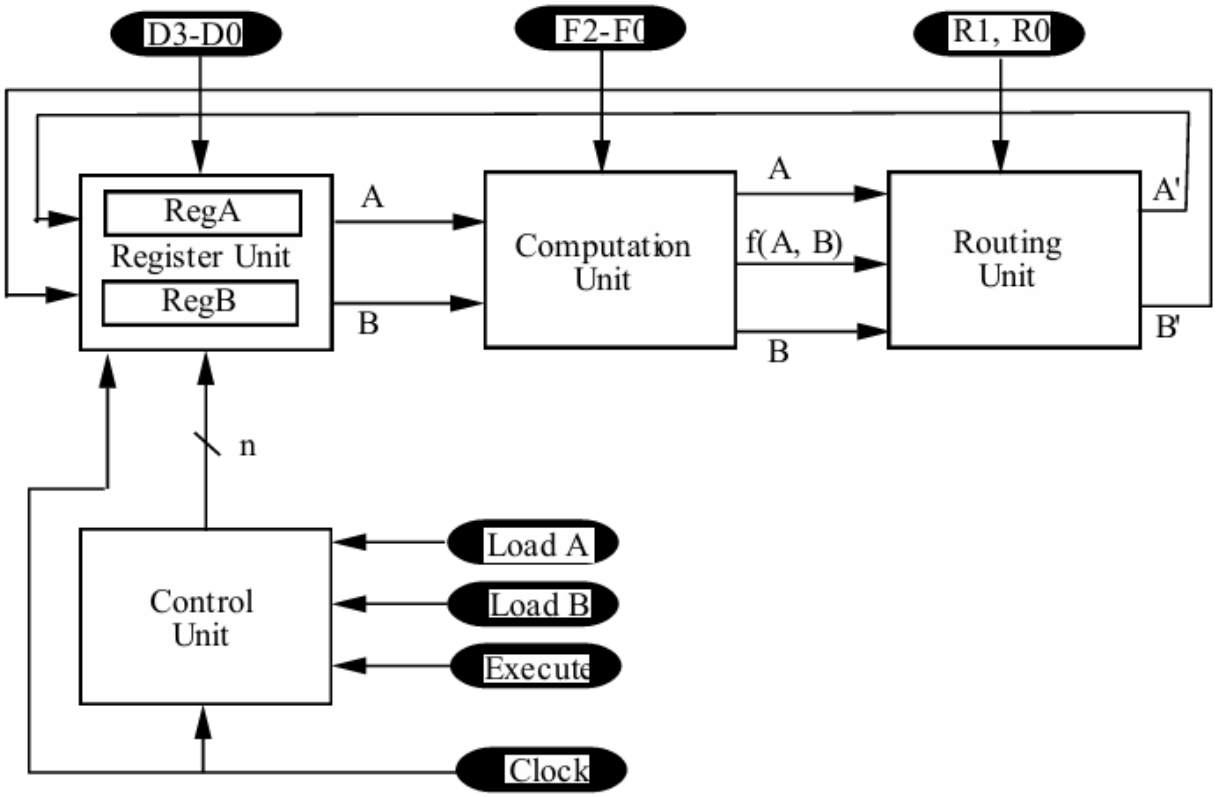

Figure 3: Block Diagram (High-level Block Diagram)

State Machine Diagram

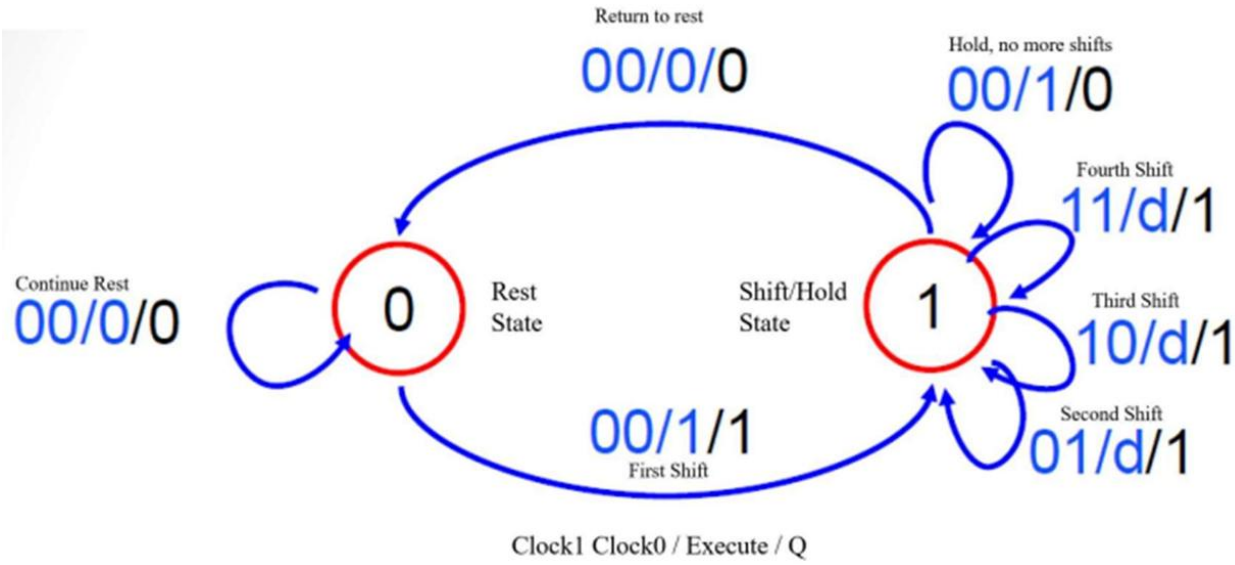

Figure 4: Mealy State Machine

Transition Arc	Description is Arc	State
00/0/0	Counter 0, Execute and S is Low	Hold in Reset State
00/1/1	Counter is 0, Execute and S is High	Shift & Compute Cycle 1
01/d/1	Counter is 1, Execute is don't care and S is High	Shift & Compute Cycle 2
10/d/1	Counter is 2, Execute is don't care and S is High	Shift & Compute Cycle 3
11/d/1	Counter is 3, Execute is don't care and S is High	Shift & Compute Cycle 4
00/1/0	Counter is 0, Execute is High and S is Low	Hold State until E is Low
00/0/0	Counter is 0, Execute and S is Low	Return to Reset State

Table 3 : Description of states in the Mealy State Machine Diagram

There are two main states in the system: rest and shift/hold. In the rest state, the machine remains inactive, doing nothing. In the shift/hold state, the machine performs exactly three additional shifts, on top of the initial shift that occurs during the state transition. The first shift happens immediately when the execute switch is pressed, marking the transition into the shift/hold state.

Once the machine has completed the four shifts (one initial shift plus three subsequent shifts), it will remain in the shift/hold state as long as the execute switch remains high. The machine will only stop shifting once all four shifts have been completed. At that point, it holds in the shift/hold state until the execute switch is toggled again.

Design steps taken and detailed circuit schematic diagram.

Written procedure of the design steps taken

To design the logic for the next state Q+Q+, we began by constructing a Karnaugh map (K-map). The K-map was populated based on the state diagram, where the 0 and 1 selections represent the actual state transitions, and the remaining states are considered 'don't care' states.

The resulting simplified logic function after solving the K-map is:

$$Q+ = E + C0 + C1$$

This represents the next state (Q+Q+) as the logical OR of the signals EE, C0C0, and C1C1.

Next, we converted this logic function into a NAND representation. The NAND form of the function is:

$$Q+ = (E' \; C0' \; C1')'$$

This means that the output Q+Q+ is the NAND of the negations of EE, C0C0, and C1C1.

The K-map, which shows the arrangement of 0s, 1s, and don't cares, would be essential for visualizing how the state transitions and the simplification were done.

Next State Q+		C1C0			
		00	01	11	10
EQ	00	0	X	X	X
	01	0	1	1	1
	11	1	1	1	1
	10	1	X	X	X

Table 4: Q+ Karnaugh Map

The next step is to design the logic for the shift register select lines, S0S0 and S1S1. Each of the two shift registers uses different select lines, which are determined by the LoadA and LoadB signals. Since S1S1 controls the parallel load, it should be high (1) only when either LoadA or LoadB is active, depending on which shift register is being selected. At all other times, S1S1 remains low (0), as shown in the Karnaugh Map.

Select Line S1		C1C0			
		00	01	11	10
EQ	00	0	X	X	X
	01	0	0	0	0
	11	0	0	0	0
	10	0	X	X	X

Table 5: S1 Karnaugh Map

From the K-map for S1S1, we derive the following equations for the select lines:

- S1A = LoadA (for shift register A)
- S1B = LoadB (for shift register B)

These equations indicate that S1S1 will control the parallel load functionality for each register based on whether LoadA or LoadB is high.

The logic for selecting S0S0, which determines whether the register should shift, hold, or parallel load, depends on the enable, state, and counter signals. The enable signal controls whether the operation is allowed, the state dictates what action to perform, and the counter helps track the current position or stage in the process. These factors together control when the shift operation occurs, when the register holds its value, or when it performs a parallel load.

Select Line S0		C1C0			
		00	01	11	10
EQ	00	0	X	X	X
	01	0	1	1	1
	11	0	1	1	1
	10	1	X	X	X

Table 6: S0 Karnaugh Map

Using the Karnaugh map for S0S0, we derive the equation S0=EQ′+C0+C1S0 = EQ' + C0 + C1. To account for the parallel load functionality, we must consider LoadA and LoadB for the two shift registers. This leads to the equations:

- S0A = A + EQ' + C0 + C1 (for shift register A)
- S0B = B + EQ' + C0 + C1 (for shift register B)

These equations are then simplified into NAND representations:

- S0A = (A' (EQ')' C0' C1')'
- S0B = (B' (EQ')' C0' C1')'

These simplified expressions were used in the final circuit design.

The main challenge in implementing the Control Unit is the design of the Finite State Machine (FSM). To tackle this, we first constructed the Karnaugh maps according to the lab manual, and from these, we designed the corresponding logic gate circuit. The K-maps that we created based on the lab manual are shown below.

		EQ			
		00	01	11	10
C1C0	00	0	0	0	0
	01	X	1	1	X
	11	X	0	0	X
	10	X	1	1	X

a) K-map for C_1^+

		EQ			
		00	01	11	10
C1C0	00	0	0	0	1
	01	X	0	0	X
	11	X	0	0	X
	10	X	1	1	X

b) K-map for C_0^+

The equations for both the K-maps are:

$$C_1^+ = C_1 \otimes C_0$$

$$C_0^+ = EQ' + C_1C_0'$$

One key component of the Control Unit is the Load A/B functionality, as shown in Figure 8. According to the lab manual, Load A/B is responsible for loading the data bits D3-D0 into the respective shift registers (Register A or B) when the Execute signal is inactive and the corresponding Load A/B is high. Initially, we decided to use three multiplexers (MUX) to implement this design. However, upon further consideration, we realized that a simpler approach

could be used. Instead of employing MUXs for the Load A and Load B inputs, these can be directly connected to the MUX that controls the input to the registers. Although this design made it easier to understand the circuit at the beginning, we later recognized that it could be optimized for efficiency.

Circuit Schematic

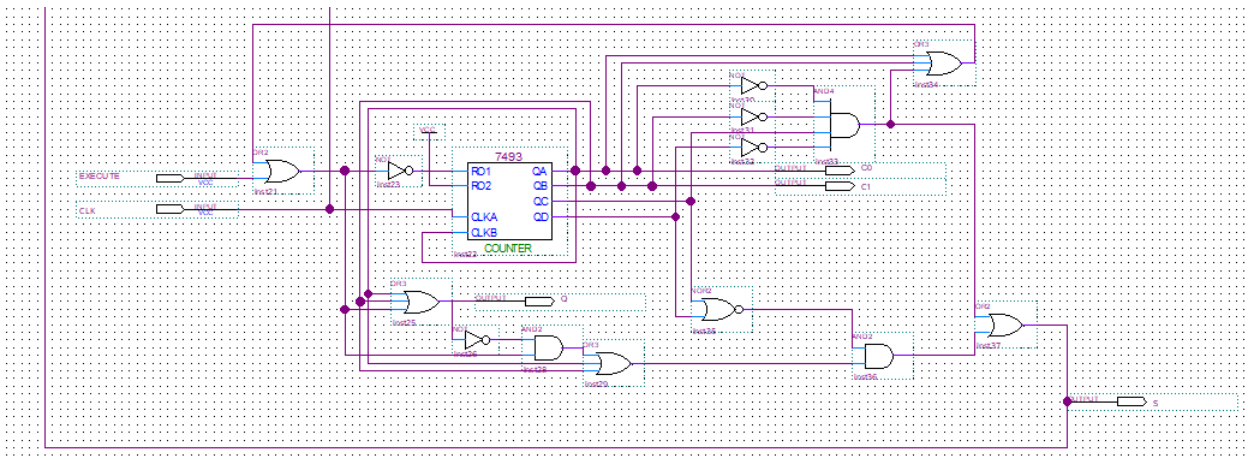

Figure 5: Control Unit

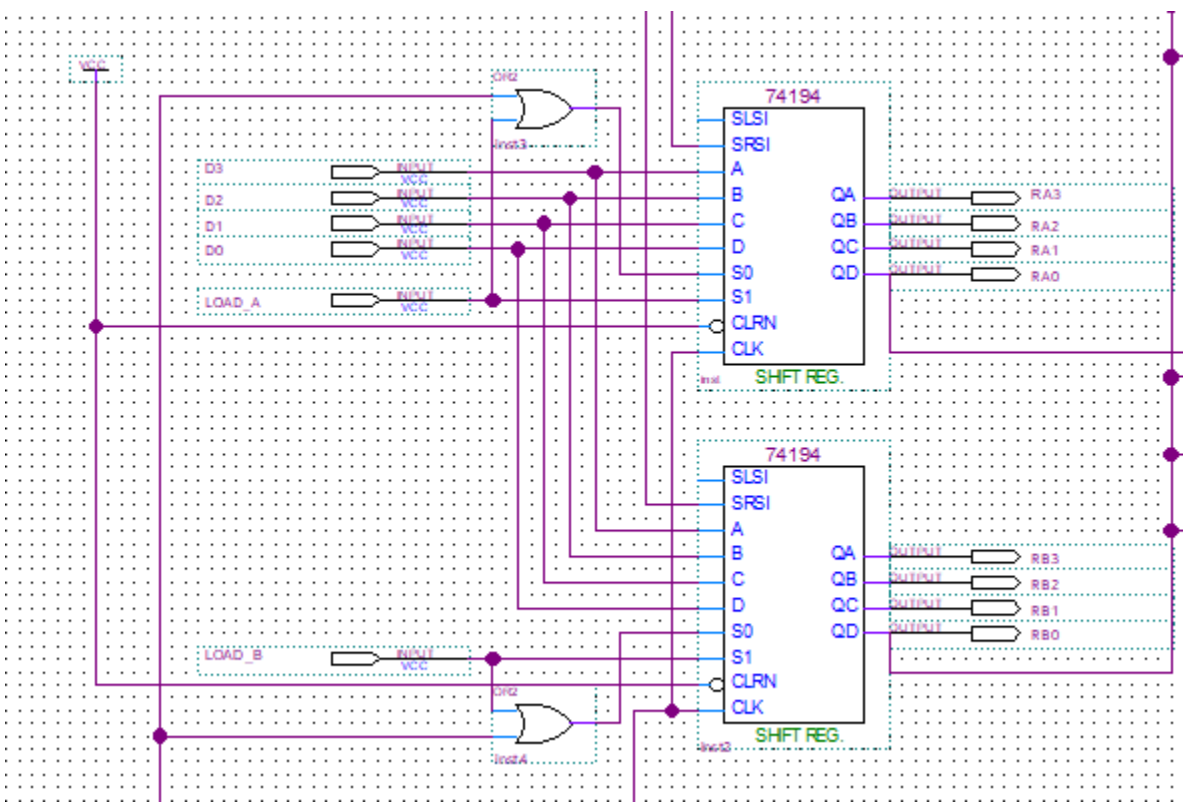

Figure 6: Load A/B diagram

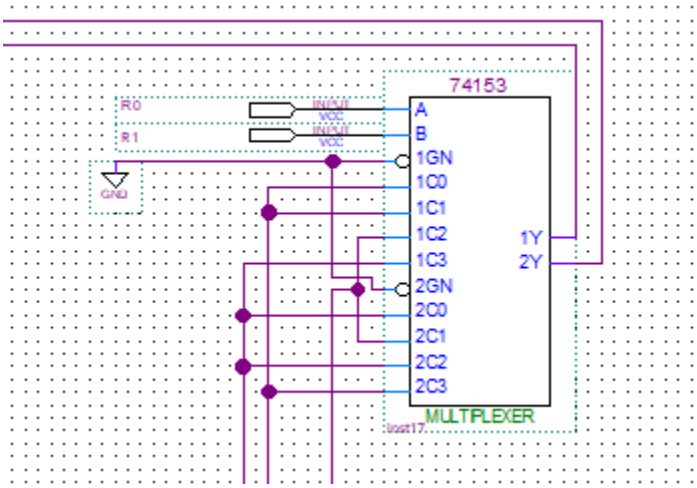

Figure 7: Routing Unit

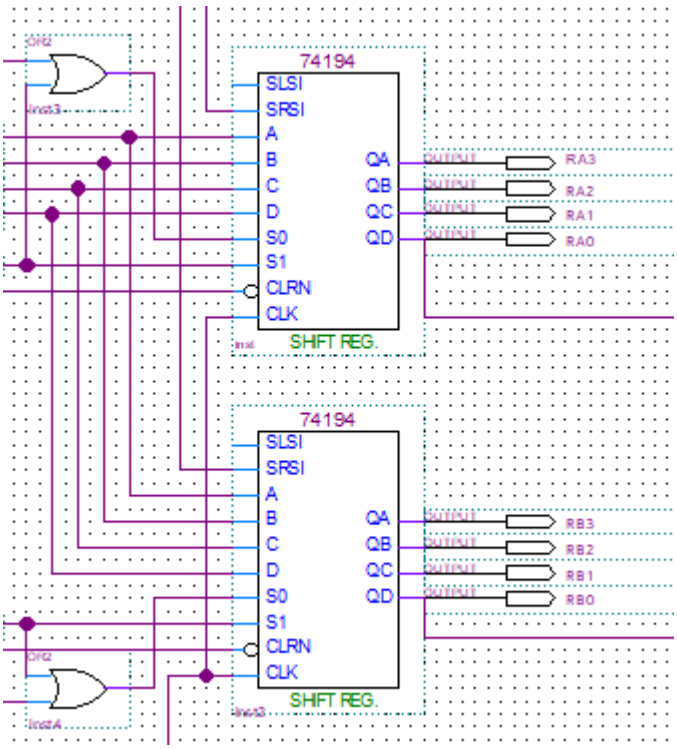

Figure 8: Register Unit

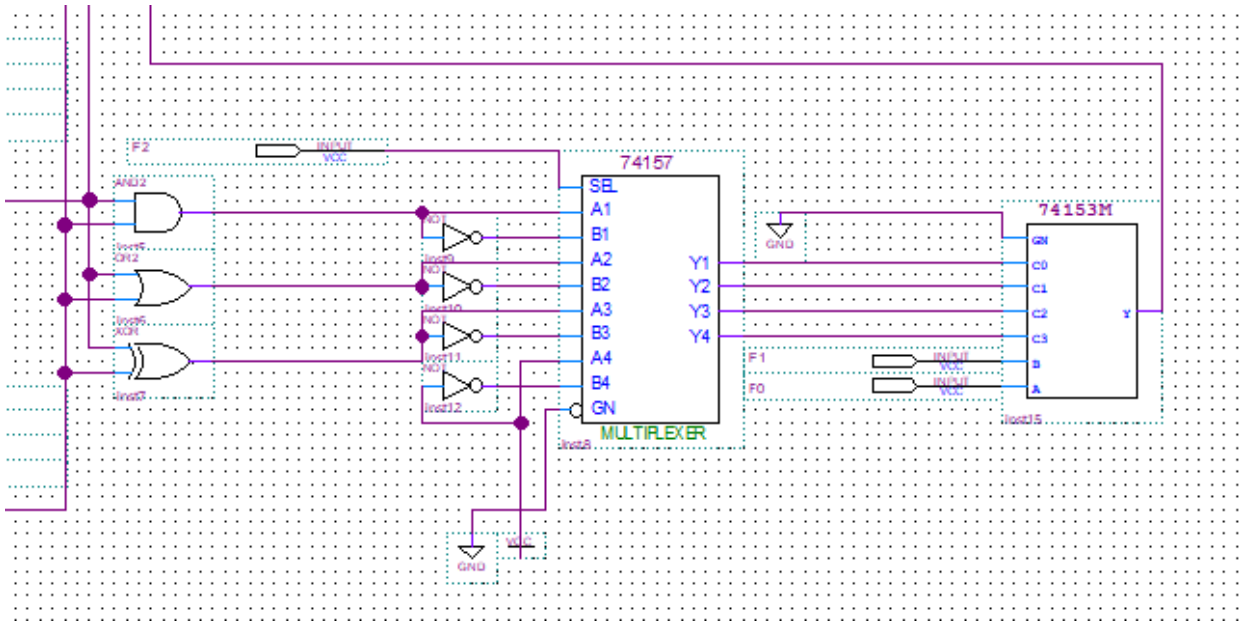

Figure 9: Computation Unit

Description of all bugs encountered, and corrective measures taken

The overall implementation of the circuit went fairly smoothly, but we encountered some challenges in understanding the specific operation of the control logic, particularly with how the Finite State Machine (FSM) interacts with the actual circuit. We struggled for a long time to understand the relationship between the S state and the E signal, as well as the connection between Load A/B and E. This confusion led us to spend a significant amount of time consulting the TA and revisiting the experimental manual during the later stages of debugging.

Initially, we faced an issue with the pin connections of the 74194 chips. We mistakenly linked the output pins incorrectly, which caused a significant deviation in our waveform simulation. After carefully inspecting the circuit, we identified the pin sequence issue and corrected it.

Additionally, we had some misconceptions about the E signal. At first, we believed that the Load signal would only function when E was high. However, we later discovered that if E is high, S will also be high. In this scenario, the priority between the Load signal and S wasn't clearly defined, which led to confusion. After discussing the issue with the TA, we realized our initial understanding was flawed.

Conclusion

Lab summarizes

In this lab, we designed and built a bit-serial logic operation processor. This processor allows us to load values into two 4-bit shift registers and perform various operations on them, such as NAND, NOR, XNOR, SET0, and more. Additionally, the routing unit enables us to select which register will store the output. To control the operations, we used a Mealy machine for the control unit, and a counter was incorporated to simplify the complexity of the control circuit.

The most challenging part of this lab was implementing the logic for the 2-state state diagram. Specifically, we faced difficulties in ensuring that the logic process executed for exactly one full rotation, regardless of whether the Execute switch was high or low during the process. This required careful attention to detail in managing the execution cycle.

Post-lab answer

Difficulties you had in debugging the circuit. How does the modular approach help you isolate design and wiring faults?

During the debugging process for the lab, we didn't encounter many significant issues, though we did experience some confusion regarding the 74LS27 chip. However, the modular design greatly helped by simplifying the wiring required for the computation unit. With simpler wiring, it became much easier to test that part of the circuit. We mainly focused on ensuring that four of the bitwise operations were functionally correct. The modular design also made it easier to verify that the entire computation unit would function properly, as it allowed us to isolate and test individual components more effectively.

What are the tradeoffs of a Mealy machine vs. a Moore machine?

- Mealy machines are more compact in terms of states and can be more responsive, but they tend to be more complex in wiring and require more components.
- Moore machines, are easier to wire and simpler in terms of understanding each state, but they often require more states to achieve the same functionality, leading to a potentially larger state machine and more states for incremental changes.